



가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 07 - Road Networks and Traffic Lab

Chris Ohk

utilForever@gmail.com

- Lecture 6의 도로망과 이동 규칙을 적용한다.
 - 건물 옆의 도로와 목적지까지 연결된 도로망을 구분한다.
- 6개의 함수를 작성하며 지난주에 만든 도시 모델을 확장한다.
 - 작성 → 단계 검사 → 화면 확인 → 조건을 바꿔 실험

- 압축을 푼 실습 폴더에서 실행한다.
 - Windows: `py -m tkinter → py app.py`
 - macOS / Linux: `python3 -m tkinter → python3 app.py`
- `traffic.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

- 제공된 코드에서 TODO 주석 부분을 채운다.

파일	역할	수업 중 수정
traffic.py	교통 관련 계산 함수	직접 작성
model.py / city_data.py	초기 도시 / 편집 / 저장	제공
app.py / traffic_gui.py / gui.py	실행 / 지도 / 버튼 표시	제공
traffic_data.py / routing.py	교통 자료와 비교용 경로 탐색	제공
checks.py	단계별 자동 검사	실행

첫 화면 살펴보기

- 도로망 분석 / 경로 비교 / 대기 차량 / 일자리 접근성을 확인한다.

도로망과 교통

기본 도시

혼잡 도시

단절 도시

도시 편집

도시 지도 / 경로 / 선택 도로의 대기열

저장

불러오기

이동: 상하좌우 / 도로 한 구간: 100 m

지도에서 선택

◆ 대기열 도로

◆ 출발 도로 S

◆ 도착 도로 T

◆ 셀 조회

0

1

2

3

4

0

1

2

3

물

H
6 / 10

P
공원

J
6 / 12

물

S

대기 0

T

F
8 / 20

도로망 분석

경로 비교

도로 노드 9개 / 연결 구간 10개 / 연결 요소 2개

알고리즘	이동거리	예상 시간	도달 여부
BFS	300 m	3분	도달
Dijkstra	300 m	3분	도달
A*	300 m	3분	도달

Dijkstra: (0,2) → (1,2) → (2,2) → (3,2)

일자리 접근성

시간 한도 4 분

도로 (0, 2)에서 4분 이내: 일자리 32개

계산

선택한 셀

도로 (1, 2) / 기본 통과 시간 1분

처리 용량 5대/분 / 대기 0대

선택 도로의 대기열

유입 8 대/분 처리 용량 5 대/분 대기 0 대 값 적용

1분 진행 대기열 초기화

0분 / 도로 (1, 2) / 대기 0대

예상 시간은 현재 대기열을 기준으로 계산합니다.

시간(분)	관찰 도로	유입(대)	처리(대)	남은 대기(대)
-------	-------	-------	-------	----------

건물에 인접한 도로 중 가장 빨리 도달하는 입구를 사용합니다. 같은 건물은 한 번만 셉니다.

- city는 도시 / state는 분 단위 교통 상태를 관리한다.

자료의 위치	예시	의미
city["grid"][2][1]	road = True	(1, 2)에 있는 도로 셀
city["buildings"]["J"]	capacity = 12	상점의 전체 일자리 수
state["queues"][(1, 2)]	0	이 도로의 대기 차량 수
state["capacities"][(1, 2)]	5	1분에 처리할 수 있는 대수
state["minute"]	0	교통 실험에서 흐른 분

- 좌표가 (x, y) 일 때, `grid[y][x]`를 통해 셀에 접근할 수 있다.

y / x	0	1	2	3	4
0			고립 도로		물
1	주택 H	공원 P		상점 J	물
2	S	도로	도로	T	
3	도로	도로	도로	도로	공장 F

실습 1 : 도로 그래프

- `build_road_graph(city)`를 작성한다.
 - 도로 좌표를 키로, 인접한 도로 좌표를 리스트로 저장한다.
- 고립된 도로도 빈 이웃 목록을 가진 노드다.

상하좌우 도로만 연결하기

- `neighbors4(city, x, y)`는 이미 제공되어 있다.
- 확인 순서 : 왼쪽 → 오른쪽 → 위 → 아래

도로 좌표	기록할 이웃
(2, 0)	[]
(0, 2)	[(1, 2), (0, 3)]
(1, 2)	[(0, 2), (2, 2), (1, 3)]

- `traffic.py`의 TODO 1을 완성해 보자.
 - `python checks.py --stage 1`
 - 기본 지도의 경우, 노드는 9개이고 `graph[(2, 0)]`는 빈 리스트다.
- 도로가 하나도 없다면 무엇을 반환해야 할까?

- 도로 셀마다 키를 만들고, 상하좌우의 도로만 이웃으로 추가한다.

```
def build_road_graph(city):  
    graph = {}  
  
    for y, row in enumerate(city["grid"]):  
        for x, cell in enumerate(row):  
            if cell["road"]:  
                graph[(x, y)] = []  
  
                for nx, ny in neighbors4(city, x, y):  
                    if city["grid"][ny][nx]["road"]:  
                        graph[(x, y)].append((nx, ny))  
  
    return graph
```

- `connected_components(graph)`를 작성한다.
 - 각 노드에 연결 요소 번호를 기록한다.
- 한 출발지에서 도달한 노드에는 같은 번호를 붙인다.

새 출발지에서 탐색 다시 시작

- 대기열에서 노드를 꺼내고, 아직 번호가 없는 이웃을 등록한다.

탐색 시작	발견한 도로	붙일 번호
(2, 0)	고립 도로 1개	0
(0, 2)	연결된 도로 8개	1

- `traffic.py`의 TODO 2를 완성해 보자.
 - `python checks.py --stage 2`
 - 기본 지도의 경우, 연결 요소는 2개이다.
 - 빈 그래프는 {}를 반환한다.
- 이미 번호가 있는 노드에서 다시 탐색하면 어떤 문제가 생길까?

연결 요소 해설 : 새 출발지

- 아직 번호가 없는 노드에서 탐색을 시작하고 같은 번호를 붙인다.

```
def connected_components(graph):  
    components = {}  
    component_id = 0  
  
    for start in graph:  
        if start in components:  
            continue  
  
        queue = deque([start])  
        components[start] = component_id
```

연결 요소 해설 : 같은 번호 기록

- 처음 발견한 이웃에 번호를 붙이고, 탐색이 끝나면 번호를 올린다.

```
while queue:
    current = queue.popleft()

    for neighbor in graph[current]:
        if neighbor not in components:
            components[neighbor] = component_id
            queue.append(neighbor)

    component_id += 1

return components
```

실습 3 : BFS 경로

- `bfs_path(graph, start, goal)`을 작성한다.
 - 경로를 좌표 리스트로 반환하고, 도달할 수 없으면 `None`을 반환한다.

parent로 지나온 경로 기억하기

- 처음 발견한 이웃의 parent에 현재 노드를 기록한다.

발견한 노드	parent
(0, 2)	None
(1, 2)	(0, 2)
(2, 2)	(1, 2)
(3, 2)	(2, 2)

- `traffic.py`의 TODO 3을 완성해 보자.
 - `python checks.py --stage 3`
 - 기본 경로 : $(0, 2) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (3, 2)$
 - 출발과 도착이 같은 도로면 `[start]` / 경로가 없으면 `None`이다.
- 출발지나 목적지가 도로가 아니라면 무엇을 반환할까?

BFS 경로 해설 : 탐색 시작

- 출발지를 큐에 넣고, parent의 키로 처음 발견한 노드를 구분한다.

```
def bfs_path(graph, start, goal):  
    if start not in graph or goal not in graph:  
        return None  
  
    queue = deque([start])  
    parent = {start: None}  
  
    while queue:  
        current = queue.popleft()
```

BFS 경로 해설 : 복원과 다음 후보

- 목적지에서 parent를 따라 경로를 복원하고 순서를 뒤집는다.

```
if current == goal:
    path = []

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()
    return path

for neighbor in graph[current]:
    if neighbor not in parent:
        parent[neighbor] = current
        queue.append(neighbor)

return None
```

실습 4 : 거리와 시간

- `route_metrics(path, travel_times)`를 작성한다.
 - 반환값 : `distance_m, time_min`
- 한 칸 이동은 100m, 시간은 셀의 진입 시간 값을 더한다.

출발 셀은 비용 합계에서 제외

- `path[1:]`에 있는 셀의 진입 시간을 더한다.

합산 대상	기본	(1, 2)가 혼잡할 때
출발 셀 (0, 2)	제외	제외
(1, 2) / (2, 2) / (3, 2)	1 + 1 + 1	5 + 1 + 1
경로 전체	300m / 3분	300m / 7분

- `traffic.py`의 TODO 4를 완성해 보자.
 - `python checks.py --stage 4`
 - 기본 경로는 300m / 3분이다.
- 셀은 4개인데 왜 300m일까? 경로가 없으면 무엇을 반환할까?

- 간선 수에 100m를 곱하고, 출발 셀 다음부터 진입 시간을 더한다.



```
def route_metrics(path, travel_times, cell_distance=100):  
    if not path:  
        return None  
  
    total_time = 0  
  
    # 출발 셀은 이미 도착한 상태이므로, 다음 셀부터 진입 비용을 더한다.  
    for node in path[1:]:  
        total_time += travel_times[node]  
  
    return {"distance_m": (len(path) - 1) * cell_distance,  
            "time_min": total_time}
```

실습 5 : 대기 차량

- `advance_queue(queue, arrivals, capacity)`를 작성한다.
 - 반환값 : `served / queue`

처리한 차량과 남은 차량

- 처리 가능 대수와 도착한 대수를 구분한다.
- $served = \min(\text{대기} + \text{도착}, \text{처리 가능 대수})$

구간	시작 대기	새 도착	통과	다음 대기
0~1분	0	8	5	3
1~2분	3	8	5	6

- `traffic.py`의 TODO 5를 완성해 보자.
 - `python checks.py --stage 5`
 - 대기 0대 + 도착 8대 / 용량 5대 → 통과 5대 / 대기 3대
- 도착한 차량이 2대뿐인데 5대를 처리하면 어떤 오류가 생길까?

- 대기와 도착을 합한 뒤 실제로 처리한 대수만 뺀다.



```
def advance_queue(queue, arrivals, capacity):  
    waiting = queue + arrivals  
    served = min(waiting, capacity)  
    return {"served": served, "queue": waiting - served}
```

실습 6 : 일자리 접근성

- `count_accessible_jobs`를 작성한다.
 - 입력 : `jobs / travel_time_by_node / limit_minutes`
- 최소 이동시간이 저장된 딕셔너리는 제공 코드에서 계산해 전달한다.
 - 시간 한도 안에 도달하는 건물의 일자리 수를 합한다.

출입구가 여러 개여도 한 번만

- 출입구 하나라도 제한 시간 이내이면 그 건물을 포함한다.

건물	각 출입구까지의 시간	일자리	4분 이내에 포함
J	3분 / 4분	12개	12개
F	4분	20개	20개
J와 F 합계	—	32개	32개

- `traffic.py`의 TODO 6을 완성해 보자.
 - `python checks.py --stage 6`
 - 4분 이내 일자리 : 기본 32개 / 혼잡 20개
 - 정확히 4분인 출입구도 포함하고, 건물은 한 번만 센다.
- 같은 건물에 출입구가 여러 개 있으면 어떻게 처리할까?

일자리 접근성 해설 : 건물별 확인

- 이미 합산한 건물 ID를 기록해 중복 집계를 막는다.



```
def count_accessible_jobs(jobs, travel_time_by_node, limit_minutes):  
    total = 0  
    counted = set()  
  
    for building in jobs:  
        if building["id"] in counted:  
            continue
```

일자리 접근성 해설 : 도달하면 합산

가상공간과 메타버스
Road Networks and Traffic Lab

- 출입구 하나에 도달하면 일자리를 더하고 다음 건물로 넘어간다.

```
for entrance in building["entrances"]:
    if entrance in travel_time_by_node:
        if travel_time_by_node[entrance] <= limit_minutes:
            total += building["jobs"]
            counted.add(building["id"])
            break

return total
```

비교 실험 1 : 기본 도로망

- 기본 시나리오를 불러온 뒤 세 알고리즘을 비교한다.

방법	거리	시간
BFS	300m	3분
다익스트라	300m	3분
A*	300m	3분

비교 실험 2 : 같은 길의 혼잡

- 혼잡 시나리오는 (1, 2)에 대기 차량 20대를 둔다.
- 진입 시간 = 기본 1분 + 20대 ÷ 5대/분 = 5분

방법	거리	시간	선택
BFS	300m	7분	혼잡한 도로 통과
다익스트라	500m	5분	아래쪽 도로로 우회
A*	500m	5분	아래쪽 도로로 우회

혼잡할 때 선택한 이동 경로

- 같은 출발지와 목적지에서 경로와 시간을 함께 읽는다.

도로망과 교통

기본 도시

혼잡 도시

단절 도시

도시 편집

도시 지도 / 경로 / 선택 도로의 대기열

저장

불러오기

이동: 상하좌우 / 도로 한 구간: 100 m

지도에서 선택

◆ 대기열 도로

◆ 출발 도로 S

◆ 도착 도로 T

◆ 셀 조회

0

1

2

3

4

0

1

2

3

물

H
6 / 10

P
공원

J
6 / 12

S

대기 20

T

I
8 / 20

S: 출발 / T: 도착 / 붉은 테두리: 대기열을 관찰할 도로

H: 주택 / J: 상점 / F: 공장 / P: 공원

선택 도로의 대기열

유입

8 대/분

처리 용량

5 대/분

대기

20 대

값 적용

0분 / 도로 (1, 2) / 대기 20대

1분 진행

대기열 초기화

예상 시간은 현재 대기열을 기준으로 계산합니다.

시간(분)	관찰 도로	유입(대)	처리(대)	남은 대기(대)
-------	-------	-------	-------	----------

도로망 분석

경로 비교

도로 노드 9개 / 연결 구간 10개 / 연결 요소 2개

알고리즘	이동거리	예상 시간	도달 여부
BFS	300 m	7분	도달
Dijkstra	500 m	5분	도달
A*	500 m	5분	도달

Dijkstra: (0,2) → (0,3) → (1,3) → (2,3) → (3,3) → (3,2)

일자리 접근성

시간 한도

4 분

계산

도로 (0, 2)에서 4분 이내: 일자리 20개

선택한 셀

도로 (1, 2) / 기본 통과 시간 1분

처리 용량 5대/분 / 대기 20대

건물에 인접한 도로 중 가장 빨리 도달하는 입구를 사용합니다. 같은 건물은 한 번만 셉니다.

BFS 경로와 가장 빠른 경로

- 혼잡해져도 BFS는 간선 수가 적은 경로를 찾는다.
 - 300m / 7분인 길보다 500m / 5분인 길이 빠를 수 있다.
- 제공된 `routing.py`로 다익스트라와 A*를 비교한다.

다익스트라 : 비용이 줄면 갱신

- 더 작은 누적 시간을 찾으면 비용과 parent를 함께 바꾼다.

```
for neighbor in graph[current]:  
    new_cost = cost + travel_times[neighbor]  
  
    if new_cost < distances.get(neighbor, float("inf")):  
        distances[neighbor] = new_cost  
        parent[neighbor] = current  
        priority = new_cost + heuristic(neighbor)
```

A* : 예상 남은 시간

- A*는 $g + h$ 가 작은 후보를 먼저 확인한다.
 - h = 맨해튼 거리 $\times$ 도로 진입 시간의 최솟값

방법	다음 후보의 우선순위	공통 갱신 규칙
다익스트라	g	새 g 가 작을 때 기록 갱신
A*	$g + h$	새 g 가 작을 때 기록 갱신

비교 실험 3 : 대기 차량 누적

- 기본 시나리오에서 도착 8대 / 용량 5대로 1분씩 진행한다.

진행한 시간	대기 차량	선택 도로 진입 시간
1분	3대	1.6분
2분	6대	2.2분
3분	9대	2.8분
4분	12대	3.4분

비교 실험 4 : 도로가 끊기면

- (2, 2)와 (2, 3)의 도로를 제거한 시나리오를 확인한다.

항목	기본	단절
도로 노드 수	9	7
연결 요소 수	2	3
$S \rightarrow T$ 경로	있음	없음
4분 이내 일자리	32개	0개

연결되어 있어도 접근성은 달라진다

- 혼잡 전후에 4분 이내 일자리 수를 비교한다.

건물	일자리	기본 최소 시간	혼잡 최소 시간
상점 J	12개	3분 / 포함	5분 / 제외
공장 F	20개	4분 / 포함	4분 / 포함
4분 이내 합계	—	32개	20개

- 도시 편집에서 도로 한 칸을 추가하거나 삭제한다.
 - 편집 후 교통 지도에 적용을 누른다.
- 도로망 분석 / 경로 비교 / 일자리 접근성을 다시 계산한다.
 - 변경 전후의 결과를 예상하고 실제 결과와 비교한다.

- 도로 셀 → 그래프 → 연결 요소와 경로 → 시간 → 접근성으로 연결했다.
 - BFS는 간선 수 / 다익스트라와 A*는 주어진 이동시간을 최소화한다.
- 대기 차량과 처리 용량이 바뀌면 같은 도로의 이용 가치도 달라진다.
- `python checks.py`로 전체 결과를 확인해 보자.

감사합니다.

utilForever@gmail.com

<https://github.com/utilForever>

Facebook, Twitter, Instagram, LinkedIn: utilForever